

CS201 Lab 7

Data Structures and Algorithms

Shreya Agarwal

Rachit Nimavat

Spring 2026

This lab builds directly upon your `IntVector` library from previous labs. You will need your `vector.h`, `vector.c`, and `Makefile`. If your implementation of `IntVector` was buggy, you may start fresh by downloading a copy from the [CS201 labs Github](#).

Part 1: Best Trade

You are given daily stock prices of a company in an array. To make money in the stock market, you want to buy low and sell high. It is easier said than done, especially when you cannot go back in time. However, in this lab, we will cheat a little by looking at the entire array of stock prices to determine the best possible trade you could have made.

Given an array `int price[n]` of daily stock prices, find the maximum profit you could have achieved by making at most one buy-sell transaction. Note that you must buy the stock before you sell it. There is a trivial $O(n^2)$ solution by checking all possible pairs of buy-sell days. Optimize it to $O(n)$ time making a single pass through the array.¹

Example: Input: [100, 80, 50, 60, 70, 40, 90]. Output: 50 by buying @40 and selling @90.

Part 2: Unlimited Trades

Now you are allowed to make as many buy-sell transactions as you like. However, you cannot engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again). Given an array `int price[n]` of daily stock prices, find the maximum profit you could have achieved in $O(n)$ time.²

Example: Input: [100, 80, 50, 60, 70, 40, 90]. Output: 70 by buying @50 and selling @70 followed by buying @40 and selling @90.

Part 3: Reversing IntVector

Our `IntVector` supports `append()`, `search()`, `pop_index()`, and `pop_node()` operations with a doubly linked list as a backend. In this part, implement the following two functions:

- `IntVector *reverse(IntVector *vec)`: Reverses the order of elements in the `IntVector`. Ensure that your implementation uses $O(1)$ additional space and has a time complexity of $O(n)$, where n is the number of elements in the `IntVector`.

¹Keep track of the minimum price and best trade observed so far as you iterate through the array, and calculate the potential profit at each step.

²Imagine the stock graph as a series of mountains. Find *every* profitable transaction that exists. Traverse the array and identify every pair of local minima (valley) and local maxima (peak). If you buy at every valley and sell at every peak, you get the maximum possible profit.

- `void snap(IntVector *vec)`: Removes every second element from the `IntVector`, effectively halving its size. For example, if the `IntVector` contains `[1, 2, 3, 4, 5]`, after calling `snap()`, it should contain `[1, 3, 5]`. Ensure that your implementation has a time complexity of $O(n)$ and uses $O(1)$ additional space. Verify that you handle edge-cases of empty vectors and vectors with an odd number of elements correctly.

Questions to Ponder Over this Week

1. How would you formally prove the correctness of your algorithm for Part 1? Usually such proofs involve identifying and maintaining certain invariants. The invariant in your case is that at the end of i -th iteration, you have correctly computed the maximum profit that can be achieved by making a single buy-sell transaction in the subarray `price[0..i]`. Write a formal induction proof based on this invariant.
2. As the complexity of your algorithm increases, it becomes more challenging to ensure its correctness. It is distinct from the correctness of its implementation. How did you convince yourself that your algorithm for Part 2 is correct before you started implementing it?
3. How do you verify that your doubly linked list implementation is correct? What kind of test cases did you write to ensure its correctness?
4. How do you verify that you are not ‘leaking’ (not appropriately freeing) memory? Look up tools like `leaks` on MacOS (`man leaks`) or `valgrind` on Linux (`man valgrind`) to help you with this. Type `make check` to compile and run your test program under `valgrind`, which will report any memory leaks it detects.